

Gardener F5 Application Load Balancer Extension

Part 3 - Service (type=LoadBalancer) Detailed Design

Implementation-oriented low-level design

Version 1.0 | July 2026

Document Purpose and Scope

This part defines the complete reconciliation design for Kubernetes Services with `spec.type=LoadBalancer`. It continues the architectural foundation and internal desired-state model established in Parts 1 and 2. The focus is implementation: what the controller reads, what it validates, how it resolves backends, which CMP resources it creates, how it converges after change or failure, and how it safely deletes owned resources.

Normative design rule

Kubernetes objects are the source of desired intent. CMP/F5 resources are managed outputs. Controllers do not directly orchestrate provider calls; they delegate to model builders and deployment managers.

Reading Guide

- “Service” means a Kubernetes core/v1 Service with `spec.type=LoadBalancer`.
- “LB Service” means the CMP top-level application load-balancer service resource.
- “Virtual Server” means the CMP/F5 listener/front-end configuration for one Service port.
- “Pool” means the backend group associated with one Virtual Server.
- “Member” means one resolved backend attached to a Pool.
- “Observed state” means resources returned by CMP discovery APIs.
- “Desired state” means the internal LoadBalancerStack built from Kubernetes input.

Table of Contents

- 3.1 Design Objectives
- 3.2 Service Reconciliation Overview
- 3.3 Kubernetes Service Analysis
- 3.4 Validation
- 3.5 Backend Discovery
- 3.6 Desired State Generation
- 3.7 Resource Mapping
- 3.8 Deployment Algorithm
- 3.9 Status Update
- 3.10 Failure Recovery
- 3.11 Delete Flow
- 3.12 Sequence Diagrams

3.13 Design Decisions

3.14 Summary and Implementation Checklist

3.1 Design Objectives

3.1.1 What Kubernetes Guarantees

Kubernetes provides an eventually consistent API and a stable Service abstraction. The extension can rely on the API server to persist intent and on EndpointSlice objects to represent the current backend set. Kubernetes does not guarantee that an external provider operation has completed when the Service object is accepted; the controller must establish that convergence itself.

Kubernetes guarantee	Design implication
Service object is durable and versioned	Every reconcile reads the latest object and treats resourceVersion conflicts as retryable.
EndpointSlice reflects selected endpoints	Backend membership is derived from EndpointSlice, not inferred by listing Pods from the selector.
Deletion is two-phase when finalizers exist	The extension can block final deletion until owned CMP resources are removed.
Status is controller-owned output	The extension patches status.loadBalancer and controller-specific conditions only after verification.
Events may be duplicated or missed	Reconciliation must be idempotent and periodic resync must be safe.

3.1.2 What the Extension Must Guarantee

- Only eligible Services are managed; non-target Services remain untouched.
- Equivalent input produces equivalent logical desired state.
- Provider mutation is ordered according to CMP resource dependencies.
- No provider resource is adopted, changed, or deleted without verified ownership.
- Multi-port Services produce deterministic listener and pool mappings.
- Backend-only changes do not recreate stable front-end resources.
- Status is not used as the source of truth for provider existence.
- Controller restart, duplicate events, and transient CMP failures converge without leaking resources.
- Deletion is dependency-aware and finalizer removal occurs only after cleanup is verified.

3.1.3 Non-goals

- Implementing general-purpose Ingress host/path routing; that belongs to Part 4.
- Managing unrelated CMP/F5 resources that were created outside this extension.
- Using Pod IPs as direct backends unless a future CMP-supported backend mode is explicitly designed.
- Encoding CMP transport concerns into controller or model-builder code.
- Providing partial support for Kubernetes features that cannot be faithfully represented in CMP.

3.2 Service Reconciliation Overview

Service Reconciliation Pipeline

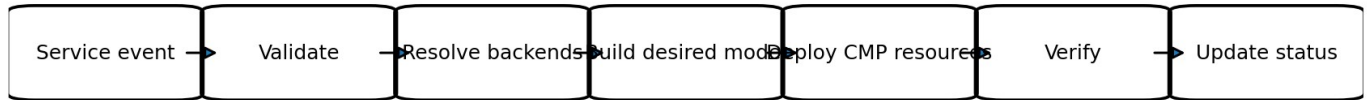


Figure 3-1. End-to-end Service reconciliation pipeline.

A reconciliation request is a request to converge, not a request to perform a particular CRUD operation. The controller always reconstructs the latest snapshot, rebuilds the full desired model, and asks deployment managers to converge observed CMP resources toward that model.

3.2.1 Controller Responsibilities

Controller does	Controller does not
Fetch Service and related Kubernetes objects	Construct raw CMP HTTP requests
Check eligibility and deletion state	Encode resource dependency ordering
Invoke validation and backend resolution	Perform ad-hoc provider retries
Invoke the model builder	Store provider state in process memory
Invoke the stack deployer	Directly create VIPs, pools, or members
Record events and patch status	Assume an event represents the complete current state

3.2.2 Reconcile Contract

```
Reconcile(ctx, NamespacedName) -> Result, error
```

1. Read the latest Service.
2. If not found: return; deletion is handled while finalizer exists.
3. If not eligible and not previously owned: return.
4. If deleting: execute cleanup path.
5. Ensure finalizer.
6. Build a consistent Kubernetes input snapshot.
7. Validate the snapshot.
8. Resolve provider-specific backend identities.
9. Build LoadBalancerStack.
10. Deploy and verify the stack.
11. Patch status and conditions.
12. Requeue only when convergence or observation requires it.

3.2.3 Event Sources

Watched object	Reason	Enqueue key
Service	Desired front-end or policy change	Service namespace/name

Watched object	Reason	Enqueue key
EndpointSlice	Backend readiness, addition, or removal	Owning Service label
Node	Node addresses, provider ID, readiness, or topology change	All affected Services
Secret or configuration object	Credential, TLS, or provider option change where supported	Referenced Services
Periodic resync	Recovery from missed events or out-of-band drift	All managed Services

3.3 Kubernetes Service Analysis

The model builder receives a normalized ServiceInput rather than a raw Service. Normalization removes API representation differences and makes all defaulting explicit. The following table identifies fields that affect the design.

Field	Used?	Design usage	Validation note
metadata.uid	Yes	Stable source identity and ownership key	Required for ownership; never derived from name alone
metadata.namespace/name	Yes	Human-readable naming and event correlation	Names may be reused after deletion; UID remains authoritative
metadata.annotations	Yes	CMP options explicitly supported by the extension	Unknown extension-prefixed annotations are rejected
metadata.deletionTimestamp	Yes	Selects cleanup flow	Mutations other than cleanup are forbidden while deleting
spec.type	Yes	Eligibility	Must equal LoadBalancer
spec.loadBalancerClasses	Yes	Controller selection	Must be absent under legacy mode or equal configured class
spec.ports	Yes	Listener, pool, member service port and health settings	Port identity must remain unambiguous
spec.selector	Indirectly	Kubernetes uses it to produce EndpointSlices	Controller consumes EndpointSlices, not Pod listing
spec.externalTrafficPolicy	Yes	Determines local or cluster backend semantics	Local requires endpoint/node locality awareness
spec.sessionAffinity	Yes when supported	Maps to CMP	Unsupported ClientIP

Field	Used?	Design usage	Validation note
		persistence profile	options are rejected
spec.loadBalancerSourceRanges	Optional	Maps to front-end source restrictions if CMP supports it	Rejected when CMP cannot represent it
spec.ipFamilies / ipFamilyPolicy	Yes for validation	Ensures selected address family is supported	No silent IPv4/IPv6 downgrade
spec.allocateLoadBalancerNodePorts	Yes	NodePort backend precondition	Must be true or defaulted true for NodePort mode
spec.healthCheckNodePort	Conditional	Local-policy health checking where applicable	Read-only input; must be allocated by Kubernetes
status.loadBalancer	Output only	Published VIP/IP or hostname	Never used to discover provider resources
spec.clusterIP / clusterIPs	No for CMP front end	Internal Service routing only	Not translated into CMP resources
spec.internalTrafficPolicy	No	Controls in-cluster traffic	Not an external LB concern

3.3.1 Service Port Identity

Each Service port must map to one stable logical listener identity. The identity is not the array index because ordering may change. The recommended logical key is: port name when present; otherwise protocol plus service port. The builder must reject duplicates after normalization.

```

servicePortKey(port):
    if port.name != "":
        return "name:" + port.name
    return lower(port.protocol) + ":" + decimal(port.port)

logicalVirtualServerID = serviceUID + "/vs/" + servicePortKey
logicalPoolID          = serviceUID + "/pool/" + servicePortKey

```

3.3.2 Annotation Design

Annotations are a compatibility surface and therefore must be few, typed, documented, and versioned. They should override optional policy, not replace core Kubernetes semantics.

Annotation category	Examples of intent	Rule
Placement	CMP project, network, subnet, availability zone	Prefer controller configuration or Gardener infrastructure context; allow override only when required
Frontend	Requested VIP, internal/external exposure	Validate immutability and address-family support
Pool	Load-balancing algorithm, connection limits	Map only to verified CMP enum values

Annotation category	Examples of intent	Rule
Health	Monitor protocol, interval, timeout, path	Defaults must be deterministic per Service protocol
Lifecycle	Deletion protection or retain-on-delete if policy allows	Must not bypass ownership checks
Observability	Optional correlation labels	Cannot alter ownership identity

3.4 Validation

Validation Pipeline

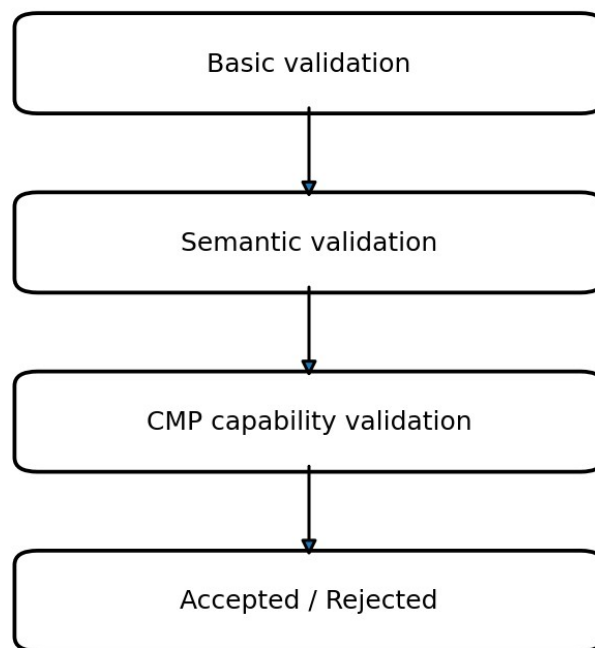


Figure 3-2. Validation stages and outcome.

3.4.1 Admission vs Reconciliation Validation

Validation type	Admission webhook	Reconciliation
Syntax and annotation parsing	Yes	Repeated defensively
Cross-field semantic checks	Yes where all input exists	Yes
Endpoint availability	No	Yes
Node to VM mapping	No	Yes
CMP capability discovery	Only static capability configuration	Yes when runtime-dependent
Ownership conflict	No	Yes
Existing provider drift	No	Yes

Why validation is repeated

Admission improves user feedback but cannot guarantee that runtime dependencies remain valid. Reconciliation therefore validates again before provider mutation. The webhook is not the sole correctness boundary.

3.4.2 Basic Validation Rules

- Service type is LoadBalancer.
- loadBalancerClass matches the configured controller class or the extension is operating in an explicitly configured legacy mode.
- At least one Service port exists.
- Protocol is supported by the current CMP virtual-server capability set.
- Every Service port has a valid nodePort in NodePort backend mode.
- IP family and requested VIP family are supported.
- Extension-prefixed annotations parse successfully and unknown keys are rejected.

3.4.3 Semantic Validation Rules

- Port logical keys are unique after normalization.
- A requested shared VIP does not create conflicting protocol/port listener tuples.
- externalTrafficPolicy=Local is accepted only when local endpoint membership can be represented correctly.
- sessionAffinity=ClientIP is accepted only when a compatible CMP persistence mode exists.
- Monitor protocol is compatible with the pool/member protocol.
- Immutable placement or VIP attributes are not changed through an ordinary update unless a controlled replacement strategy exists.
- Annotation combinations do not request both mutually exclusive internal and external exposure.

3.4.4 Validation Result Model

```
type ValidationResult struct {
    Accepted bool
    Warnings []Warning
    Errors   []FieldError
}

type FieldError struct {
    Path      string
    Code      string
    Message   string
    Retryable bool
}
```

Non-retryable configuration errors set an InvalidConfiguration condition and wait for a Kubernetes object change. Retryable dependency errors, such as a temporarily unavailable network lookup, set a ResolvingBackends or ProviderUnavailable condition and use bounded backoff.

3.5 Backend Discovery

Backend Resolution Chain

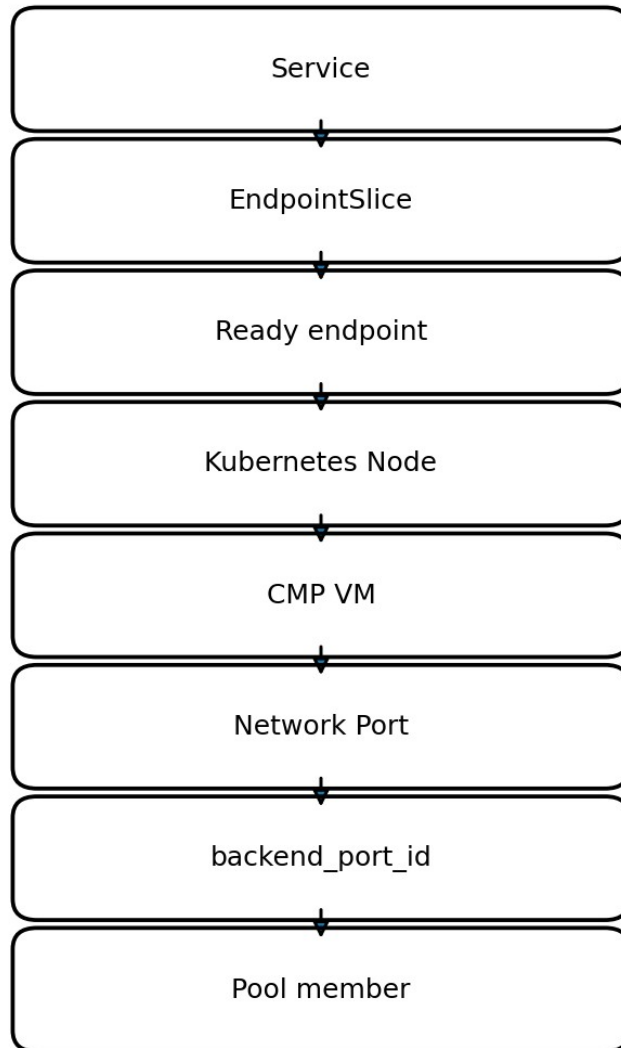


Figure 3-3. Backend resolution from Kubernetes endpoint to CMF network-port identity.

Backend discovery is intentionally isolated behind a BackendResolver interface. Kubernetes identifies application endpoints; CMF requires provider-specific compute and network identities. The resolver joins these domains without coupling the controller or model builder to CMF lookup mechanics.

3.5.1 Input Snapshot

- Normalized Service and Service ports.
- EndpointSlices selected by the standard `kubernetes.io/service-name` label.
- Nodes referenced by eligible endpoints.
- Provider/Gardener infrastructure context needed to locate CMF tenant, project, network, or region.
- Optional cached read-only CMF lookup data with explicit freshness limits.

3.5.2 Endpoint Selection Algorithm

1. List EndpointSlices for the Service and flatten endpoints by port identity.
2. Match each EndpointSlice port to the Service port by name first; use protocol and port only where Kubernetes semantics permit an unambiguous match.
3. Ignore terminating endpoints unless the configured drain policy intentionally retains them for a grace period.
4. Include Ready endpoints. Treat unknown readiness according to a documented conservative policy; default is not ready.
5. Require nodeName for NodePort backend mode.
6. For externalTrafficPolicy=Local, include a node only when that node has at least one eligible local endpoint for the Service port.
7. For externalTrafficPolicy=Cluster, deduplicate eligible nodes because kube-proxy can route NodePort traffic to cluster endpoints.
8. Sort the resulting backend candidates by stable node UID or provider ID to produce deterministic model output.

3.5.3 Node to CMP VM Resolution

```
resolveNode(node):
    require node.spec.providerID
    providerRef = parseProviderID(node.spec.providerID)
    vm = cmpComputeClient.GetByProviderReference(providerRef)
    verify vm belongs to expected project/region
    return vm.id, vm.resourceType
```

Node names are not sufficient provider identities. The primary mapping key is the provider ID or an infrastructure-specific immutable machine reference. Name-based fallback should be disabled by default because names can collide or be reused.

3.5.4 Network Port and backend_port_id Resolution

9. Collect the node internal addresses that belong to the selected backend network family.
10. Query the CMP network API by network/project and IP address, or enumerate VM-attached ports when that API is more precise.
11. Filter results to the expected network/subnet and VM ownership.
12. Require exactly one matching network port.
13. Return the CMP network port identifier as backend_port_id.
14. Treat zero matches as retryable only while infrastructure may still be converging; otherwise report BackendResolutionFailed.
15. Treat multiple matches as a non-retryable ambiguity until configuration changes.

3.5.5 Resolved Backend Model

```
type ResolvedBackend struct {
    NodeUID      string
    NodeName     string
    VMID         string
    VMResourceType string
    Address      string
}
```

```

BackendPortID  string
NodePort       int32
ServicePortKey string
Weight         int
LocalEndpointCount int
}

```

3.5.6 Caching and Consistency

Compute and network lookups may be cached because many Services share Nodes. Cache entries must be keyed by immutable provider identity plus project/network scope, must have a bounded TTL, and must be invalidated on Node provider-ID/address changes. A cache hit never bypasses ownership or network-scope verification.

Failure	Classification	Controller behavior
EndpointSlice not yet created	Retryable dependency	Set BackendsPending and requeue
No ready endpoint	Valid zero-backend state	Keep front end; deploy empty pool or preserve per policy; report NoReadyBackends
Node missing	Retryable during churn	Re-read and requeue
providerID missing	Configuration/infrastructure error	Report NodeIdentityUnavailable
VM lookup timeout	Transient provider error	Backoff and retry
Multiple network ports match	Non-retryable ambiguity	Do not mutate provider; report precise candidates
backend_port_id missing	Provider data incomplete	Retry with limit, then surface error

3.6 Desired State Generation

Service Desired-State Resource Graph

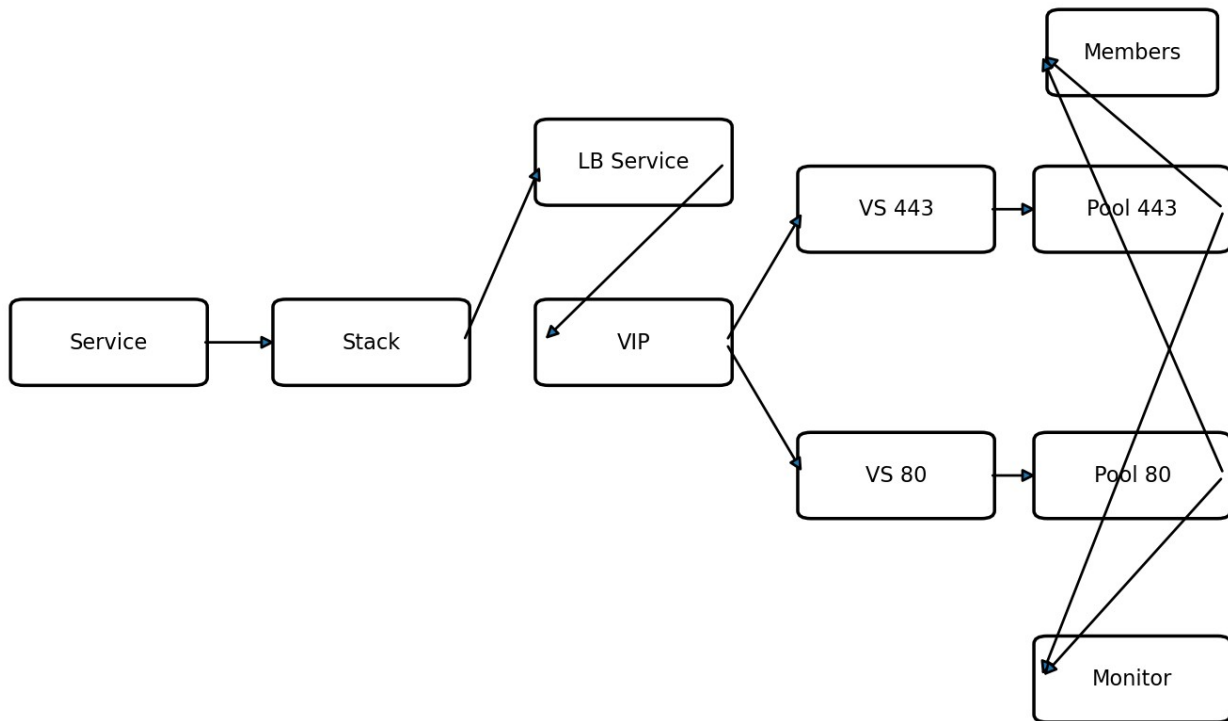


Figure 3-4. Internal desired-state graph for a two-port Service.

The ServiceModelBuilder is a pure transformation over normalized Kubernetes input and resolved backend identities. It performs no CMP calls. This makes model output deterministic, unit-testable, and reusable for dry-run or diff tooling.

3.6.1 Builder Contract

```

type ServiceModelBuilder interface {
    Build(ctx context.Context, in ServiceBuildInput) (*LoadBalancerStack, error)
}

type ServiceBuildInput struct {
    Service    NormalizedService
    Backends   []ResolvedBackend
    Defaults   ProviderDefaults
    Scope      InfrastructureScope
}

```

3.6.2 LoadBalancerStack

Model object	Cardinality	Key content
LoadBalancerStack	1 per Service	Source identity, ownership, graph metadata
LBServiceSpec	1	Placement, display name, description, tags

Model object	Cardinality	Key content
VIPSpec	1 normally	Requested or allocated address, family, exposure, subnet
VirtualServerSpec	1 per Service port	Protocol, listener port, VIP reference, pool reference, persistence
PoolSpec	1 per Service port	Algorithm, member references, monitor reference
MemberSpec	0..N per pool	backend_port_id, address, NodePort, weight
MonitorSpec	0..1 per pool or shared compatible monitor	Protocol, interval, timeout, request/expected response

3.6.3 Logical IDs and Ownership Metadata

Logical IDs are internal stable identities used to correlate desired objects with observed provider resources. Provider IDs are discovered outputs and must not be embedded in the desired model as identity.

```

stackLogicalID = "service/" + serviceUID
lbServiceLogicalID = stackLogicalID + "/lb-service"
vipLogicalID     = stackLogicalID + "/vip/default"
virtualServerID  = stackLogicalID + "/virtual-server/" + portKey
poolLogicalID    = stackLogicalID + "/pool/" + portKey
memberLogicalID  = poolLogicalID + "/member/" + nodeUID
monitorLogicalID = poolLogicalID + "/monitor/default"

```

Ownership field	Produced by	Purpose
sourceKind	Model builder	Kubernetes source type, e.g. Service
sourceNamespace	Model builder	Human and query correlation
sourceName	Model builder	Human correlation
sourceUID	API server, copied by builder	Authoritative source identity
controllerID	Controller configuration	Distinguishes controller installation
logicalID	Model builder	Correlates desired and observed resource
managedBy	Model builder constant	Marks extension ownership domain
role	Model builder per resource	lb-service, vip, virtual-server, pool, member, monitor

Who creates ownership fields?

The Kubernetes API server creates metadata.uid. The model builder creates the ownership envelope for every desired CMP resource using that UID, controller configuration, and the resource role. Deployment managers serialize this envelope into CMP-supported tags, metadata,

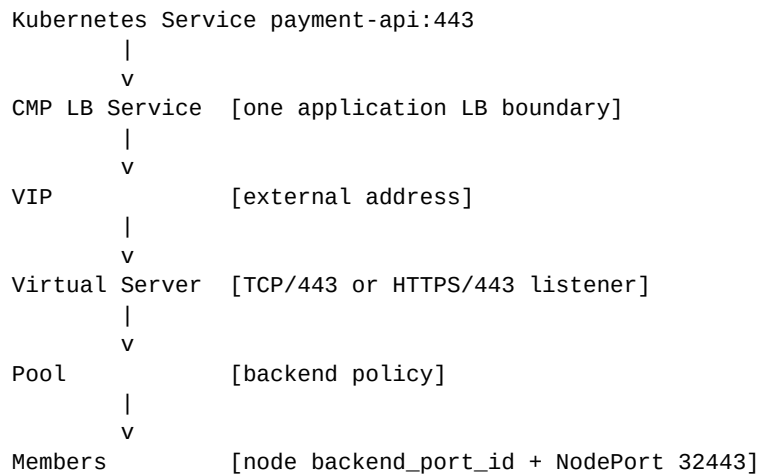
description fields, or an external tracking record. Typed CMP clients do not invent ownership.

3.6.4 Deterministic Defaults

- Default pool algorithm is centrally configured and materialized into the model.
- Default health monitor is selected from protocol-aware policy.
- Absent optional annotations resolve to explicit model values rather than remaining implicit.
- Lists are sorted by logical ID before deployment and comparison.
- Names are generated from a bounded, collision-resistant scheme containing source name plus UID fragment.

3.7 Resource Mapping

3.7.1 Single-port Service



The CMP LB Service is intentionally present. It is the top-level provider resource under which the VIP, virtual server, pools, and related resources are created. A Kubernetes Service is not mapped directly to a VIP alone.

3.7.2 Multi-port Service

Kubernetes port	Virtual Server	Pool	Members
http:80 -> nodePort 30080	VS http-80 on shared VIP	Pool http-80	Eligible nodes at 30080
https:443 -> nodePort 30443	VS https-443 on shared VIP	Pool https-443	Eligible nodes at 30443
metrics:9100 -> nodePort 30100	VS tcp-9100 on shared VIP when allowed	Pool tcp-9100	Eligible nodes at 30100

The default mapping is one Virtual Server and one Pool per Service port, while the LB Service and VIP are shared. This prevents protocol, health-monitor, and backend-port settings from becoming entangled across ports.

3.7.3 Resource Replacement Boundaries

Change	Expected action
Backend membership changes	Update pool members only
Health monitor parameters change	Update or replace monitor; keep VIP and listener stable
Pool algorithm changes	Update pool in place when CMP supports it
Service listener port changes	Create new Virtual Server/Pool, verify, then delete old pair
Requested VIP changes	Controlled VIP/front-end replacement; do not silently mutate if CMP treats address as immutable
Placement/project/network changes	Usually full stack replacement under explicit policy
Service UID changes with same name	Treat as a new owner; old resources belong to deleted source and are cleaned independently

3.8 Deployment Algorithm

Deployment Algorithm

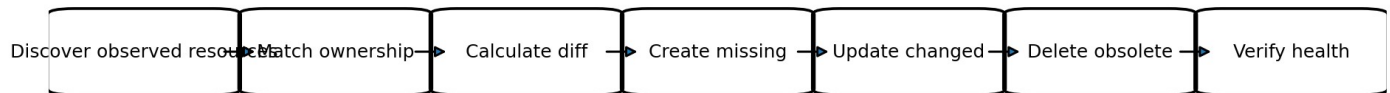


Figure 3-5. Dependency-aware deployment algorithm.

Deployment managers implement resource-specific convergence behind a StackDeployer. The deployer builds an operation plan from desired and observed resource graphs, then executes dependency-safe actions. CMP API calls are made only through typed clients.

3.8.1 Discovery

16. Query CMP resources using the strongest available ownership selector, preferably sourceUID plus controllerID.
17. If CMP lacks server-side metadata filtering, query within the infrastructure scope and filter locally using normalized ownership metadata.
18. Build an ObservedStack containing provider IDs, normalized mutable fields, health, and dependency references.
19. Detect duplicate observed resources with the same logical ID and stop automatic mutation until conflict policy resolves them.
20. Do not use Kubernetes status as proof that a provider resource exists.

3.8.2 Matching and Diff

```

for each desired resource D:
    O = observedByLogicalID[D.logicalID]
    if O absent: plan CREATE(D)
    else if !ownershipMatches(D, O): plan CONFLICT
    else if immutableFieldsDiffer(D, O): plan REPLACE(O, D)
    else if mutableFieldsDiffer(D, O): plan UPDATE(O.id, patch(D,O))
    else: plan NOOP

for each observed resource O not matched by desired:
    if ownershipMatchesStack(O): plan DELETE(O)

```

3.8.3 Create Order

21. LB Service
22. VIP
23. Monitors that do not require pools
24. Pools
25. Pool members
26. Virtual Servers and routing references after their dependencies exist
27. Provider-specific associations or activation operations
28. Readiness verification

Exact ordering must follow CMP API constraints discovered from the Swagger specification. Where CMP creates nested resources through a parent endpoint, the deployment manager still exposes the same logical dependency boundary to the stack deployer.

3.8.4 Update and Delete Order

Operation	Ordering principle
Backend addition	Create member then verify pool health
Backend removal	Drain if supported, then remove member
Listener replacement	Create new pool/listener, verify, then remove old resources
Obsolete child deletion	Members before pool; listener references removed before pool deletion
Full cleanup	Virtual Servers/rules/certificates -> members -> pools/monitors -> VIP -> LB Service
Shared-resource deletion	Delete only when no remaining owned desired resource references it

3.8.5 Idempotency and Retries

- Create uses idempotency keys when CMP supports them; otherwise discovery follows an ambiguous timeout before retrying create.
- Update sends only normalized mutable fields to avoid provider-generated field churn.
- Delete treats not-found as success.

- HTTP 409 or provider in-progress states cause observation and retry, not blind repeated mutation.
- Authentication failure is surfaced immediately and retried at a slower provider-availability cadence.
- Every mutation is followed by read-after-write verification when CMP is asynchronous.

3.9 Status Update

Status communicates externally observable state; it is not the provider inventory database. The status writer receives a typed deployment result and produces a minimal merge patch.

3.9.1 When Status Is Updated

Situation	Status action
Validation failed	Set Accepted=False or Ready=False with non-retryable reason; do not publish a new VIP
Backend resolution pending	Set Ready=False, reason BackendsPending
Deployment in progress	Set Ready=False, reason Provisioning; preserve existing valid ingress address during safe updates
Verified ready	Publish VIP/hostname and Ready=True
Transient CMP failure	Set Ready=False or Degraded=True while preserving last known endpoint if still observed healthy
Deletion in progress	Optionally set Deleting condition; final object may disappear before users observe it

3.9.2 Standard Service Status

```
status:
  loadBalancer:
    ingress:
      - ip: 203.0.113.42
```

The extension publishes the provider-assigned VIP as an IP or hostname according to Kubernetes ServiceStatus semantics. It must not publish an address before the CMP front end is verified usable according to the selected readiness policy.

3.9.3 Conditions

Condition	Meaning
Accepted	The Service configuration is supported and valid
BackendsResolved	All required Kubernetes-to-CMP backend identities were resolved
Provisioned	Required CMP resources exist with desired configuration
Ready	Front end is active and provider verification

Condition	Meaning
	succeeded
Degraded	Service remains partially usable but drift or provider error exists
Deleting	Owned resources are being removed

3.9.4 Conflict-safe Status Patching

- Use status subresource patching, not full-object update.
- Preserve fields written by other controllers where Kubernetes permits multiple writers.
- Do not update transitionTime when condition status and reason are unchanged.
- Retry resourceVersion conflicts by reading the latest Service.
- Emit Kubernetes Events only for meaningful transitions, not every retry.

3.10 Failure Recovery

Recovery is achieved by reconstructing desired state and re-discovering observed state on every reconcile. No correctness-critical workflow depends on in-memory progress.

Scenario	Detection	Recovery action
VIP exists, pool missing	Observed graph is incomplete	Keep owned VIP; create missing pool and remaining children
Pool exists with stale members	Desired/observed diff	Add and remove members only
Controller restarts after create timeout	Provider response lost	Discover by ownership/idempotency key before issuing another create
CMP returns asynchronous provisioning	Resource health/state not ready	Requeue with bounded polling; do not recreate
CMP timeout during update	Outcome ambiguous	Read observed resource and compare normalized fields
Out-of-band field modification	Drift detected	Restore managed fields; ignore documented provider-owned fields
Resource ownership metadata removed	Ownership verification fails	Stop destructive actions and report OwnershipConflict
Duplicate logical resources	Discovery finds >1 match	Mark conflict; apply explicit duplicate cleanup only under safe policy
Node replacement	Old node disappears, new provider ID appears	Add new member, verify, then remove old member
No ready backends	Desired member set empty	Keep or disable front end according to policy; report NoReadyBackends
Status patch fails	Kubernetes API	Provider remains converged;

Scenario	Detection	Recovery action
	conflict/unavailable	retry status patch without repeating unnecessary mutations

3.10.1 Error Taxonomy

Class	Examples	Requeue behavior
InvalidInput	Unsupported protocol, ambiguous annotations	No timed retry; wait for object change
DependencyPending	EndpointSlice absent, VM not yet visible	Short bounded retry
ProviderTransient	Timeout, 429, 5xx	Exponential backoff with jitter
ProviderAuth	Invalid credential/signature	Slow retry and prominent event/metric
Conflict	Foreign ownership, duplicate logical IDs	No destructive mutation; periodic observation
TerminalProvider	CMP rejects valid model as unsupported	Surface ProviderRejected; retry only after relevant change

3.10.2 Observability Requirements

- Reconcile duration and result counters labelled by phase, not by Service name.
- CMP request metrics by operation/resource/status code with sensitive values removed.
- Backend-resolution failure counters by reason.
- Current managed stacks, ready stacks, degraded stacks, and deletion backlog gauges.
- Structured logs containing source UID, namespace/name, logical ID, provider ID, and reconcile correlation ID.
- Kubernetes Events for user-actionable failures and successful initial provisioning.

3.11 Delete Flow

Deletion and Cleanup

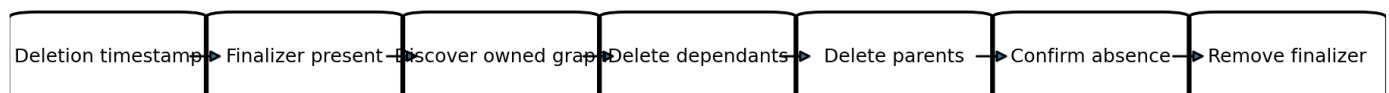


Figure 3-6. Finalizer-driven cleanup flow.

3.11.1 Finalizer Ownership

The Service controller adds a dedicated finalizer only after the Service is accepted for management and before creating CMP resources. The finalizer means “provider cleanup may be required”; it is not proof that provider resources currently exist.

```
networking.f5.gardener.cloud/service-load-balancer-finalizer
```

3.11.2 Cleanup Algorithm

29. Read the deleting Service and reconstruct its source ownership identity from metadata.uid.
30. Discover all CMP resources owned by that source UID and controller ID within the infrastructure scope.
31. Build the observed dependency graph even when current Kubernetes ports or annotations are no longer valid.
32. Delete dependants before parents. Treat not-found as success.
33. Poll asynchronous deletions until absence or a documented terminal state is observed.
34. If a resource is foreign-owned or ownership is ambiguous, stop and surface CleanupBlocked rather than deleting it.
35. After no owned resources remain, remove the finalizer with a conflict-safe patch.

3.11.3 Why Cleanup Cannot Depend Only on Current Desired State

A deleting Service may have been modified before deletion, may no longer pass validation, or may be missing referenced Kubernetes objects. Cleanup must therefore be driven by immutable source ownership and observed provider resources, not solely by rebuilding the current desired model.

3.11.4 Force and Retain Policies

Policy	Default	Design requirement
Retain provider resources on Service delete	Disabled	Allowed only as an explicit administrative policy with clear ownership transfer semantics
Force finalizer removal after timeout	Disabled	Manual operator action; automatic force risks leaks or foreign deletion
Delete protection in CMP	Honoured and reported	Controller must not silently bypass provider protection
Orphan cleanup	Separate operator workflow	Must prove ownership and infrastructure scope before deletion

3.12 Sequence Diagrams

Service Create Sequence

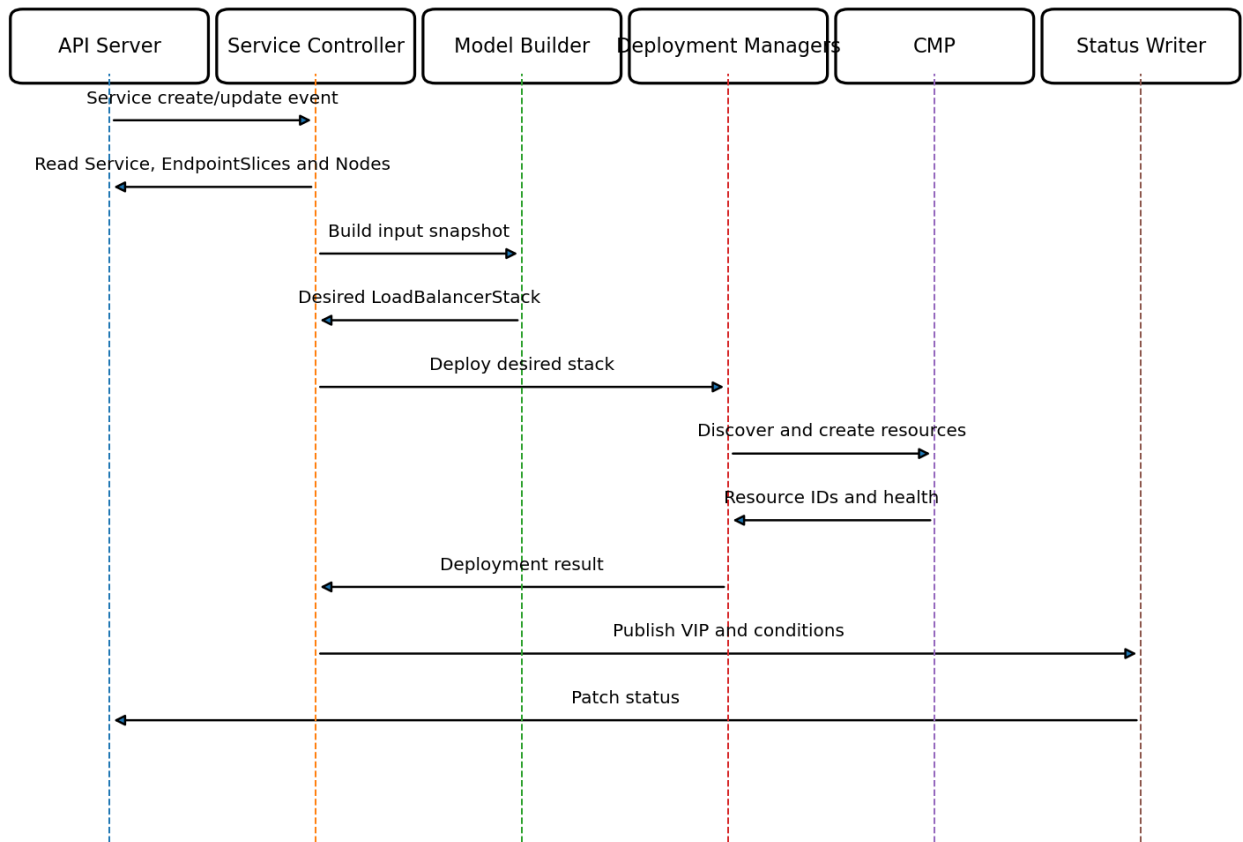


Figure 3-7. Service creation and initial provisioning.

The initial create path publishes status only after deployment managers report that required resources are present and the selected readiness check succeeds.

Service Update / Scale Sequence

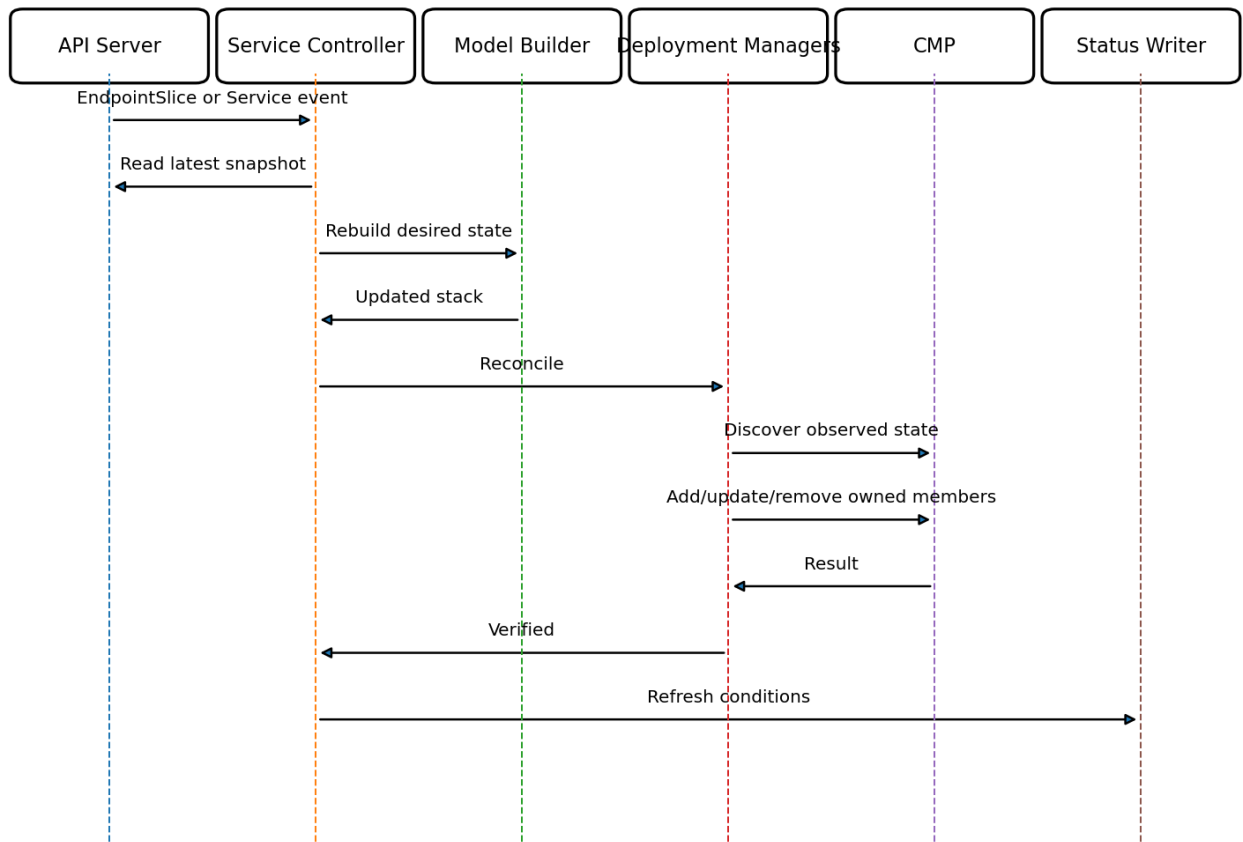


Figure 3-8. Service update, scale-up, scale-down, and backend churn.

Scale events normally change only member resources. The model is rebuilt in full, but the diff plan protects the stable LB Service, VIP, Virtual Servers, and Pools from unnecessary recreation.

Service Delete Sequence

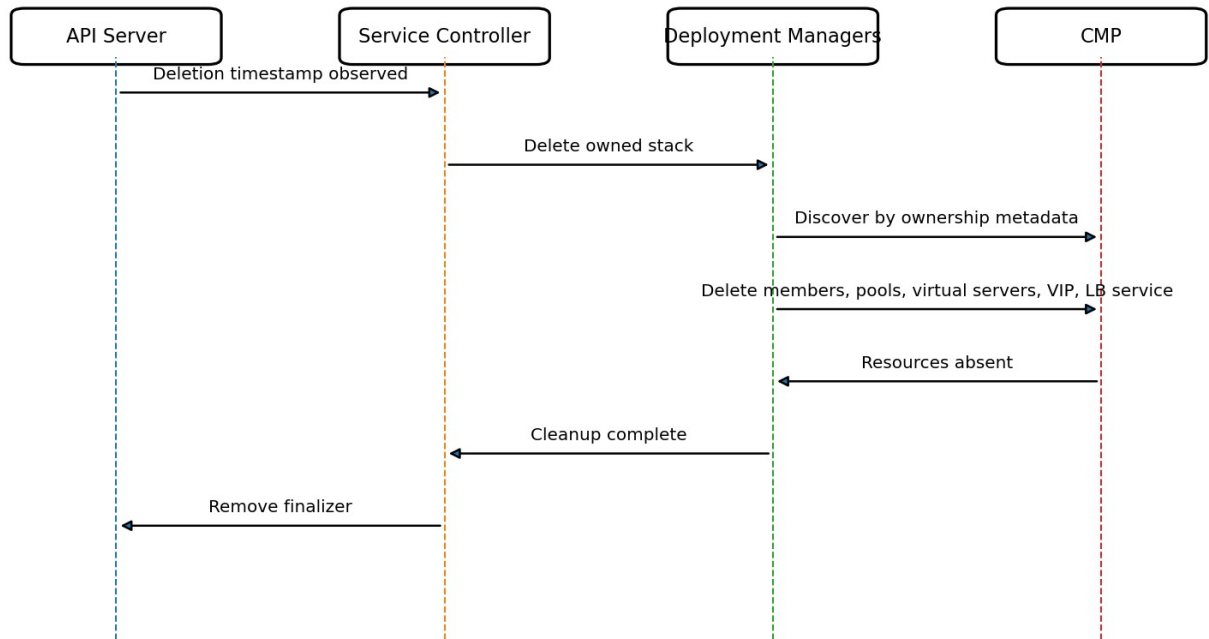


Figure 3-9. Service deletion and finalizer removal.

3.12.1 State Machine

Service Reconciliation States

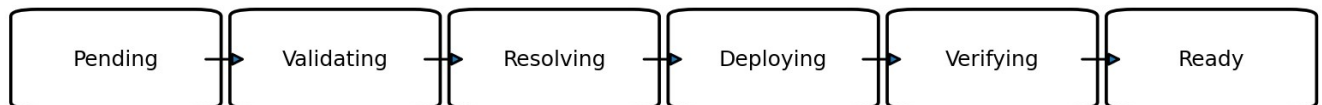


Figure 3-10. Main reconciliation states. Any state may transition to Degraded or Deleting.

State	Entry condition	Exit condition
Pending	Event received or resync	Latest object read
Validating	Eligible non-deleting Service	Validation accepted
Resolving	Kubernetes snapshot valid	Required backend identities resolved
Deploying	Desired stack built	Operation plan completed
Verifying	Resources created or updated	Readiness confirmed
Ready	Status published	New event, drift, or resync
Degraded	Retryable failure or drift	Recovery succeeds or input changes

State	Entry condition	Exit condition
Deleting	Deletion timestamp with finalizer	Owned provider graph absent

3.12.2 Node Replacement Flow

36. EndpointSlice/Node events enqueue the Service.
37. The resolver returns a new member for the replacement Node and omits the old Node when it is no longer eligible.
38. Diff creates the new member before deleting the old member when both are temporarily visible.
39. Pool verification confirms at least the required healthy capacity.
40. The old member is drained and removed.
41. No VIP, Virtual Server, or Pool replacement occurs.

3.13 Design Decisions

Decision	Rationale	Consequence
Use NodePort backends	CMP/F5 integrates with node network ports and stable node-level reachability	Requires nodePort allocation and Node-to-VM-to-port resolution
Do not use Pod IPs by default	Pod addresses are ephemeral and may not be directly reachable or representable in CMP	EndpointSlice still determines node eligibility, but member address/identity is node based
One Virtual Server per Service port	Listeners have independent protocol, port, persistence, monitor, and pool lifecycle	Multi-port Service creates multiple listener/pool pairs
One Pool per Service port	Backend NodePort and health semantics differ by port	Backend updates remain isolated per port
Share LB Service and VIP across compatible ports	Matches one Kubernetes Service external identity and avoids unnecessary addresses	Port/protocol conflicts must be validated
Backend resolution outside controller	Separates Kubernetes orchestration from provider identity lookup	Resolver can be unit tested and cached independently
Pure model builder	Enables deterministic tests and clear desired/observed diff	All provider reads happen before or within deployment layer, not during model construction
Ownership based on Service UID	Namespace/name can be reused	Safe adoption, drift repair, and deletion require immutable identity
Validation in webhook and reconcile	Webhook cannot validate runtime provider state	Correctness survives webhook bypass and changing dependencies

Decision	Rationale	Consequence
Status after verification	Provider create acknowledgement may be asynchronous	Users see an endpoint only when it is operational by defined policy

3.13.1 externalTrafficPolicy

Value	Backend model
Cluster	One member per eligible worker node; kube-proxy may forward to endpoints on other nodes
Local	Only nodes with an eligible local endpoint are members; preserves source IP where the data path supports it

The exact data-plane behavior must be confirmed for the cluster CNI, kube-proxy replacement, and F5 configuration. The controller must not claim source-IP preservation unless the complete path supports it.

3.13.2 Session Affinity

Kubernetes ClientIP session affinity may map to a CMP/F5 source-address persistence profile. Support is conditional on verified CMP API fields. If the API cannot configure or reference persistence safely, the Service is rejected rather than silently ignoring sessionAffinity.

3.13.3 Health Monitoring

- TCP is the safe default for generic TCP Service ports.
- HTTP/HTTPS monitors require explicit path, expected status/body policy, and TLS behavior.
- For externalTrafficPolicy=Local, health semantics must ensure nodes without local endpoints are not considered healthy members.
- Monitor interval and timeout must satisfy CMP validation constraints and avoid aggressive load.
- Application-specific monitor settings belong in typed annotations or a future policy CRD, not arbitrary free-form JSON.

3.13.4 Security Decisions

- CMP credentials are read from Kubernetes Secrets and never copied into desired models, status, events, or logs.
- Typed clients own Ce-Auth/HMAC signing and TLS configuration.
- Controller RBAC is restricted to required Services, EndpointSlices, Nodes, Events, Secrets/configuration, and status/finalizer updates.
- Provider operations are scoped to the configured CMP tenant/project/network.
- Ownership metadata is validated before every destructive mutation.
- User-provided names and annotations are sanitized before sending to CMP.

3.14 Summary and Implementation Checklist

The Service design translates one Kubernetes LoadBalancer Service into one owned CMP load-balancer stack. The stable top-level resources are an LB Service and normally one VIP. Each Service port receives an independent Virtual Server and Pool. EndpointSlice determines eligible backends, while a dedicated resolver converts node identity into CMP VM and backend_port_id values. Deployment managers converge this graph using ownership-aware discovery, typed clients, ordered operations, verification, and finalizer-based cleanup.

3.14.1 Package-Level Implementation Checklist

Area	Required implementation
controllers/service	Reconciler, watches, predicates, finalizer flow, event/status coordination
service/validation	Basic, semantic, annotation, capability validators
service/backend	EndpointSlice selection, Node policy, VM/port resolver, cache
service/model	Normalizer, logical IDs, ownership envelope, LoadBalancerStack builder
deploy/stack	Discovery, graph matching, diff planning, ordered execution, verification
deploy/lbservice	LB Service manager
deploy/vip	VIP manager
deploy/virtualserver	Virtual Server manager
deploy/pool	Pool and member manager
deploy/monitor	Health monitor manager
cmp/client	Typed clients, Ce-Auth/HMAC transport, retries, error normalization
status	Conditions, ServiceStatus patching, events
observability	Metrics, structured logs, trace/correlation IDs
webhooks	Admission validation and annotation schema

3.14.2 Minimum Test Matrix

Test level	Essential cases
Unit - validation	Supported/unsupported protocols, duplicate ports, annotations, policy combinations
Unit - builder	Single port, multi-port, empty backends, Local/Cluster, deterministic ordering, ownership fields
Unit - resolver	Ready filtering, node deduplication, provider ID parsing, zero/multiple port matches, cache invalidation
Unit - diff	Create, update, replace, delete, no-op, ownership conflict, duplicate observed resources

Test level	Essential cases
Integration - fake CMP	Initial create, timeout ambiguity, async provisioning, partial resource graph, drift repair
Envtest/controller	Watches, finalizer, status conflict, deletion while invalid, EndpointSlice churn
End-to-end	Service create/update/delete, multi-port, scale up/down, node replacement, controller restart, CMP outage

3.14.3 Definition of Done

- All supported Service fields have explicit mapping and validation behavior.
- No controller code directly calls raw CMP endpoints.
- Every provider resource contains verifiable ownership metadata.
- Reconciliation is idempotent under duplicate events and ambiguous provider responses.
- Scale and endpoint changes modify members without recreating the front end.
- Deletion removes the complete owned graph before finalizer removal.
- Status and events clearly distinguish invalid input, pending dependencies, provider failures, conflicts, and readiness.
- Unit, integration, and end-to-end tests cover the minimum matrix above.

Appendix A - Core Interfaces

```

type BackendResolver interface {
    Resolve(ctx context.Context, snapshot KubernetesSnapshot) ([]ResolvedBackend, error)
}

type StackDeployer interface {
    Deploy(ctx context.Context, desired *LoadBalancerStack) (DeploymentResult, error)
    Delete(ctx context.Context, owner OwnershipKey, scope InfrastructureScope) error
}

type CMPVirtualServerClient interface {
    List(ctx context.Context, scope Scope, filter OwnershipFilter) ([]VirtualServer, error)
    Get(ctx context.Context, id string) (*VirtualServer, error)
    Create(ctx context.Context, req CreateVirtualServerRequest, idempotencyKey string)
(*VirtualServer, error)
    Update(ctx context.Context, id string, req UpdateVirtualServerRequest) (*VirtualServer,
error)
    Delete(ctx context.Context, id string) error
}

```

Appendix B - Example Normalized Desired Model

```

stack:
  logicalID: service/3b6d...-uid
  owner:
    sourceKind: Service
    sourceNamespace: payments
    sourceName: payment-api
    sourceUID: 3b6d...-uid
    controllerID: gardener-extension-f5/default
    managedBy: gardener-f5-alb-extension
  lbService:
    logicalID: service/3b6d...-uid/lb-service
  vip:
    logicalID: service/3b6d...-uid/vip/default
    family: IPv4
  virtualServers:
  - logicalID: service/3b6d...-uid/virtual-server/name:https
    protocol: TCP
    port: 443
    poolRef: service/3b6d...-uid/pool/name:https
  pools:
  - logicalID: service/3b6d...-uid/pool/name:https
    algorithm: ROUND_ROBIN
    members:
    - backendPortID: cmp-port-7e3a
      address: 10.20.1.14
      port: 30443

```